

El Caldero Chorreante (Gestión de Pociones)

Resumen

Con esta práctica se pretende afianzar de forma progresiva los conceptos de programación orientada a objetos (POO) en Java. Implementarás una aplicación de consola que permita interactuar con un inventario de pociones mágicas en Hogwarts. En lugar de usar librerías externas, empezarás construyendo tu propia herramienta de lectura de datos.

FASE 1: Construcción de la Herramienta de Lectura (Teclado.java)

Antes de empezar a fabricar pociones, necesitas un instrumento mágico para interactuar con el usuario desde la consola: la clase `Teclado`. El objetivo es crear una herramienta reutilizable y robusta que impida que el programa se rompa si el usuario introduce datos incorrectos.

Paso 1: Configuración del Paquete e Importación

Crea un fichero llamado `Teclado.java`. En la primera línea debes definir que pertenece al paquete de utilidades: `package com.hogwarts.utils;`. Justo debajo, importa la herramienta nativa de Java para escanear la consola: `import java.util.Scanner;`.

Paso 2: El Escáner Oculto y Estático

Dentro de la clase `Teclado`, define un atributo privado y estático de tipo `Scanner`. Al hacerlo `static`, compartiremos el mismo lector en toda la aplicación:

```
private static final Scanner scanner = new Scanner(System.in);
```

Paso 3: Método Paso a Paso para Leer Cadenas (String)

Crea un método público y estático llamado `leerCadena`. y que retorne lo que escribe el usuario por consola.

Qué hace: Debe recibir como parámetro un texto (`String mensaje`) que sirva para indicar al usuario qué debe escribir (por ejemplo: *"Introduce el ingrediente: "*) y que retorna lo que escribe el usuario en la consola.

Lógica: Imprime el mensaje en la pantalla y devuelve directamente lo que capture el escáner usando `scanner.nextLine()`;

Paso 4: Método Robusto para Leer Números Enteros (`int`)

Crea un método público y estático llamado `leerEntero`. Este método debe ser "a prueba de errores". Si se pide un número y el usuario introduce una letra, el programa no debe cerrarse con un error, sino avisar y volver a preguntar.

Qué hace: Recibe un `String mensaje` y devuelve un `int`.

Lógica:

1. Crea una variable entera `numero = 0` y un booleano `valido = false`.
2. Abre un bucle `while (!valido)`.
3. Muestra el mensaje por pantalla.
4. Utiliza una condición: `if (scanner.hasNextInt())`. Esto "espía" la consola. Si lo que viene es un número, lo absorbe con `numero = scanner.nextInt()`; y cambia `valido = true`.
5. Si no es un número (`else`), muestra un mensaje de advertencia (ej: *"¡Eso no es un número entero válido!"*) y limpia el dato erróneo usando `scanner.next()`;
6. **Crucial:** Justo antes de terminar el método (fuera del bucle), añade un `scanner.nextLine()`; . Esto limpia el "Intro" o salto de línea residual que queda en la memoria del sistema para que no falle en futuras lecturas de texto.
7. Devuelve el `numero`.

FASE 2: Clases, Objetos y Ocultación (Encapsulamiento)

2.1. Clase Poción

Implementa la clase `Pocion` dentro del paquete `com.hogwarts.laboratorio`.

Atributos:

- `private int ingredientesRestantes`: Cantidad que le falta a la poción para estar lista (atributo de instancia).
- `private int ingredientesIniciales`: Almacena la cantidad inicial para poder reiniciar el proceso si el caldero falla.

- `private static int totalPocionesCreadas`: Atributo estático (de clase). Lleva la cuenta global de cuántas pociones se han creado en total en el programa. Empezará en 0.

Métodos:

- `Constructor`: Acepta un entero con los ingredientes necesarios. Inicializa `ingredientesRestantes` e `ingredientesIniciales` con ese valor, e incrementa en 1 el contador estático `totalPocionesCreadas`.
- `public void muestraEstado()`: Imprime por pantalla cuántos ingredientes faltan por añadir.
- `public boolean añadirIngrediente()`: Resta 1 a `ingredientesRestantes`. Devuelve `true` si todavía le faltan ingredientes. Si llega a 0, muestra el mensaje *"¡Poción Elaborada con Éxito!"* y devuelve `false`.
- `public void vaciarCaldero()`: Reinicia la poción asignando a `ingredientesRestantes` el valor original de `ingredientesIniciales`.
- `public static void muestraTotalCreadas()`: Método estático que muestra el contenido del contador global.

2.2. Clase Main

Crea la clase `Main` en el paquete `com.hogwarts.main`. En su método `main`:

1. Comprueba que no puedes modificar `ingredientesRestantes` directamente desde fuera de la clase debido al modificador `private`.
2. Instancia una poción requiriendo 3 ingredientes.
3. Añade dos ingredientes usando los métodos de la poción y muestra su estado.
4. Vacía el caldero y comprueba que vuelve a requerir los 3 ingredientes iniciales.
5. Crea una segunda poción y llama al método estático `Pocion.muestraTotalCreadas()` para verificar que el contador global marca 2.

FASE 3: Herencia y Polimorfismo

El laboratorio de Hogwarts exige especialización. La clase `Pocion` pasará a ser una **clase abstracta**.

3.1. Modificación de la Clase Pocion

- Añade la palabra clave `abstract` a la definición de la clase.
- Añade el método abstracto: `public abstract void removerCaldero();` (sin llaves, termina en punto y coma).

3.2. Nueva Clase: PociónCurativa

- Debe crearse en el paquete `com.hogwarts.recetas` y heredar (`extends`) de `Poción`.
- Su constructor recibe los ingredientes y se los pasa al padre mediante `super(ingredientes)`.
- Implementa `removerCaldero()`: Las pociones curativas son muy inestables al removerse; este método debe llamar automáticamente a `vaciarCaldero()` para arruinar el progreso e imprimir: *"¡El caldero se ha desestabilizado! El proceso vuelve a empezar."*

3.3. Nueva Clase: PociónVeneno

- Debe crearse en el paquete `com.hogwarts.recetas` y heredar de `Poción`.
- Añade el atributo `private String colorVapor`.
- Su constructor recibe los ingredientes y el color del vapor.
- Implementa `removerCaldero()`: Imprime *"El caldero desprende un peligroso vapor de color [colorVapor]"* y añade automáticamente un ingrediente llamando internamente a `añadirIngrediente()`.

FASE 4: Interfaces y Colecciones Dinámicas (ArrayList)

Para poder operar de manera uniforme con cualquier creación alquímica, utilizaremos un contrato de comportamiento.

4.1. Interfaz Alquímica

Crea la interfaz `Alquimica` dentro del paquete `com.hogwarts.interfaces` con los métodos:

- `void preparar();`
- `void infoIngredientes();`

4.2. Implementación del Contrato

Haz que `PociónCurativa` y `PociónVeneno` implementen (`implements`) la interfaz `Alquimica`.

- El método `preparar()` llamará internamente al método `removerCaldero()` correspondiente de cada clase y mostrará el estado resultante.
- El método `infoIngredientes()` mostrará un resumen de la poción actual.

4.3. Uso de ArrayList en Aplicación

Modifica la clase `Aplicacion` utilizando la colección dinámica `ArrayList` de `java.util`:

1. Crea un método estático `public static Alquimica elegirPocion()`.
2. Dentro de este método, instancia un `ArrayList<Alquimica>` y añádele una `PocionCurativa` y una `PocionVeneno` mediante el método `.add()`.
3. Muestra un menú utilizando tu clase de utilidad: `Teclado.leerEntero(...)`.

¿Qué receta deseas preparar en el caldero?

1. Poción Curativa (Filtro de Paz)
2. Poción de Veneno (Vapores nocivos)

4. El método debe extraer el objeto seleccionado utilizando `.get(indice)` (recuerda restar 1 a la opción del usuario para cuadrar el índice del array) y devolverlo como el tipo genérico de la interfaz: `Alquimica`.
5. En el `main`, recupera esa poción y ejecútala repetidamente en un bucle utilizando el método `.preparar()` hasta que se complete su elaboración.

FASE 5: Gestión de Excepciones

Las reacciones mágicas con compuestos oscuros pueden provocar desastres que el sistema debe controlar.

5.1. Clase `PocionInestableException`

Crea esta clase dentro del paquete `com.hogwarts.excepciones`. Debe extender de la clase nativa `Exception`. Su constructor recibirá un mensaje (`String`) describiendo la anomalía.

5.2. Validación en el Constructor de Venenos

Modifica el constructor de la clase `PocionVeneno`. Si al crear la poción el parámetro `colorVapor` contiene algún número (por ejemplo, "`Morado7`"), el constructor **debe lanzar** (`throw new`) una `PocionInestableException` con el mensaje: *"Error de Alquimia: Los vapores oscuros no pueden contener magnitudes numéricas"* (puedes valerte de un bucle y el método `Character.isDigit()` para analizar la cadena).

5.3. Estructura de Control en Aplicacion

Modifica el método `main` de tu clase `Aplicacion` para envolver la selección y preparación en un bloque de seguridad `try-catch`.

- Si se captura una `PocionInestableException`, muestra el mensaje de error de forma clara por pantalla.
- Añade la sección `finally` para asegurar que, pase lo que pase en el caldero, la consola imprima al terminar: *"Limpiando la mesa de trabajo. Fin de la clase de Pociones."*